- 3 challenges w/ processing graphs:
  1) Their topology is variable [hard to design networks that are both expressive & can cope w/ the variation]
  2) Graphs can be massive
  3) May only be a single large graph, so the normal training strategy [many data examples to train, testing w/ new data] isn't always appropriate.

13.1: What is a graph?

- Data structure consisting of a set of nodes / vertices, pairs of nodes are connected by edges.
  - Typically sparse

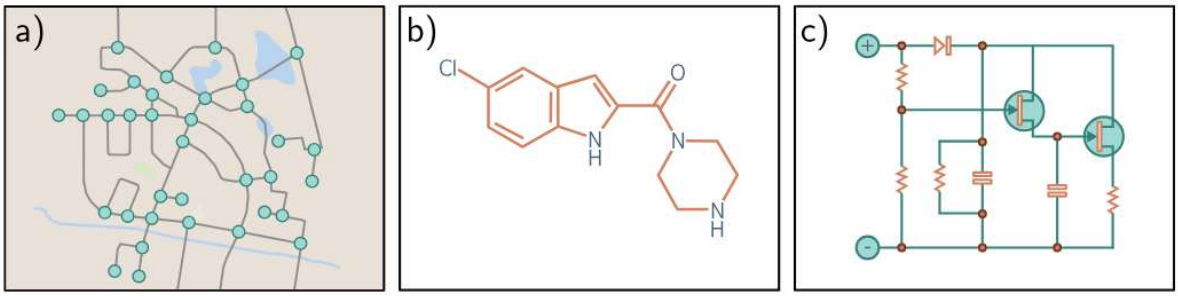


Figure 13.1 Real-world graphs. Some objects, such as a) road networks, b) molecules, and c) electrical circuits, are naturally structured as graphs.

- Graphs can be categorized in various ways:
  - Undirected
  - Directed
  - Knowledge graph
  - Geometric graph
  - Hierarchical graph

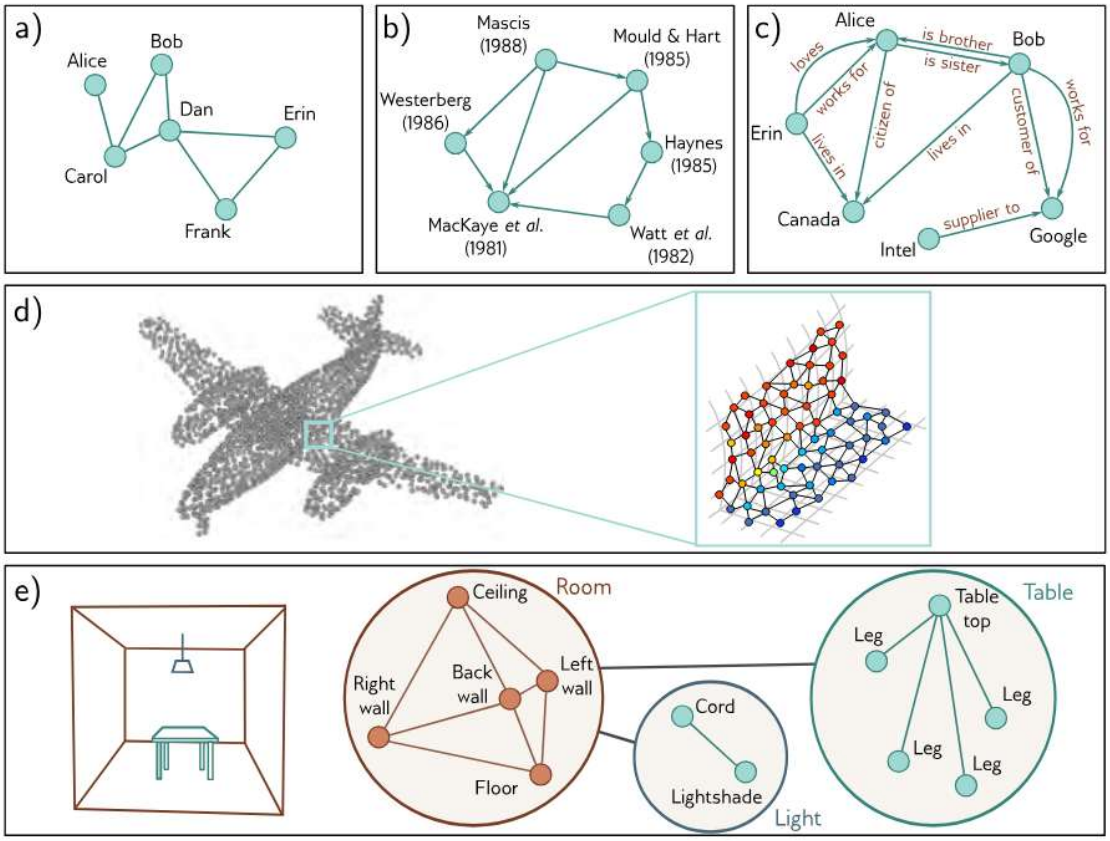


Figure 13.2 Types of graphs. a) A social network is an undirected graph; the connections between people are symmetric. b) A citation network is a directed graph; one publication cites another, so the relationship is asymmetric. c) A knowledge graph is a directed heterogeneous multigraph. The nodes are heterogeneous in that they represent different object types (people, places, companies) and multiple edges may represent different relations between each node. d) A point set can be converted to a graph by forming edges between nearby points. Each node has an associated position in 3D space, and this is termed a geometric graph (adapted from Hu et al., 2022). e) The scene on the left can be represented by a hierarchical graph. The topology of the room, table, and light are all represented by graphs. These graphs form nodes in a larger graph representing object adjacency (adapted from Fernández-Madrigal & González, 2002).

13.2: Graph Representation:

- In addition to the graph structure, information is usually associated with each node.
  - ↳ Information at a node is stored in a node embedding.
  - ↳ Information at an edge is stored in an edge embedding.

- Formally:
  - ↳ A graph has a set of N nodes connected by a set of E edges.
  - ↳ A graph can be encoded by 3 matrices:

1) A → Graph structure

2) X → Node embeddings

3) E → Edge embeddings

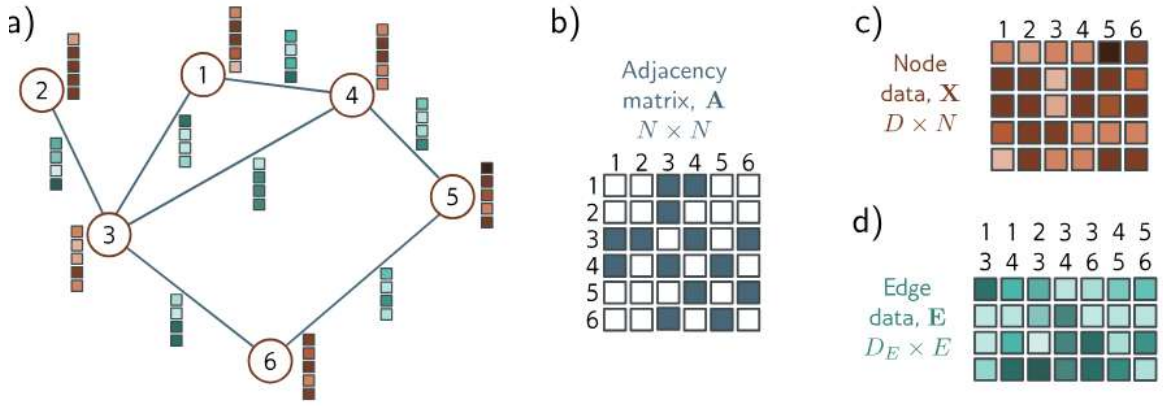


Figure 13.3 Graph representation. a) Example graph with six nodes and seven edges. Each node has an associated embedding of length five (brown vectors). Each edge has an associated embedding of length four (blue vectors). This graph can be represented by three matrices. b) The adjacency matrix is a binary matrix where element (m, n) is set to one if node m connects to node n. c) The node data matrix $\mathbf{X}$ contains the concatenated node embeddings. d) The edge data matrix $\mathbf{E}$ contains the edge embeddings.

- The graph structure is represented by an adjacency matrix. (A)
  - ↳ N x N matrix, each entry (m,n) = 1 if there's an edge between nodes m & n, 0 otherwise.
  - ↳ For undirected graphs, this matrix is symmetric.
  - ↳ For large sparse graphs, it can be stored as a list of connections (m,n) to save memory.
- The nth node has an associated node embedding $x^{(n)}$ of length D.
- These embeddings are concatenated & stored in the D x N node data matrix X.
- Similarly, the eth edge has an edge embedding $e^{(e)}$ of length D_E.
- These embeddings are collected into the D_E x E matrix E.
- The adjacency matrix can be used to find neighbors of a node using linear algebra.
  - ↳ Encoding the nth node's position as a 1 hot column vector.

↳ Encoding the nth node's position as a 1 hot column vector.

↳ Pre-multiplying this vector by the adjacency matrix extracts the nth column of the adjacency matrix ; returns a vector with 1's in the position of the neighbors.

↳ Repeating this operation, the resulting vector contains the # of walks of length 2 from node n to every node.

- In general, if we raise the adjacency matrix to the power of L, the entry at position (m,n) of A^L contains the # of unique walks of length L from node m to n.
  - ↳ Not the same as the # of unique paths since the walks include nodes that visit the same node more than once.
  - ↳ A^L still contains valuable information about graph connectivity.
    - ↳ A non-zero entry at (m,n) means the distance from m to n must be $\leq L$.

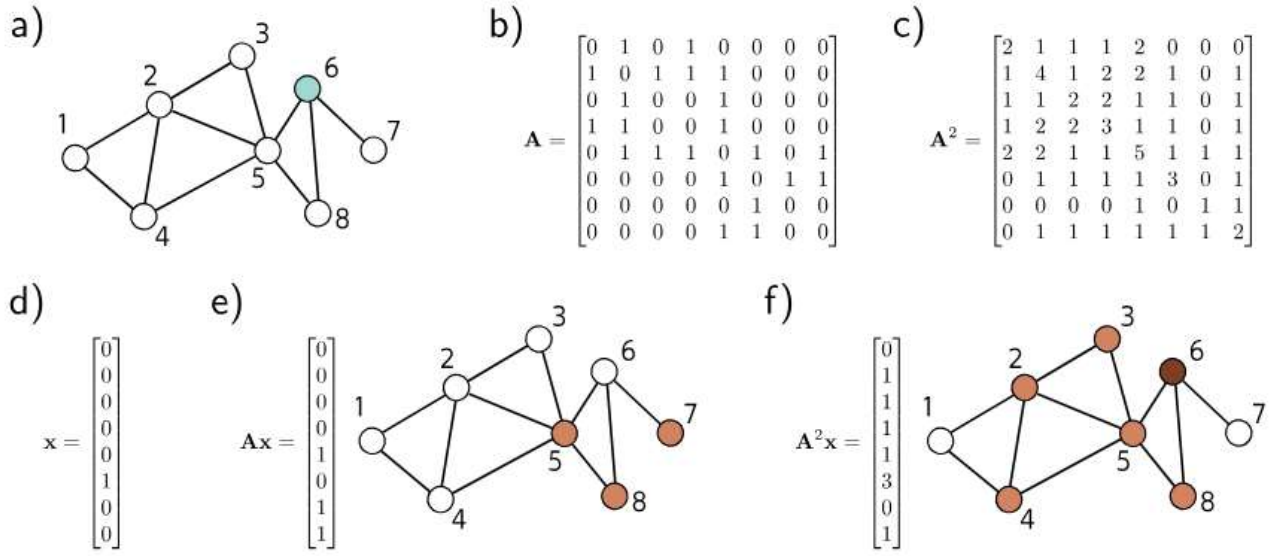


Figure 13.4 Properties of the adjacency matrix. a) Example graph. b) Position (m, n) of the adjacency matrix $\mathbf{A}$ contains the number of walks of length one from node m to node n. c) Position (m, n) of the squared adjacency matrix $\mathbf{A}^2$ contains the number of walks of length two from node m to node n. d) One hot vector representing node six, which was highlighted in panel (a). e) When we pre-multiply this vector by $\mathbf{A}$, the result contains the number of walks of length one from node six to each node; we can reach nodes five, seven, and eight in one move. f) When we pre-multiply this vector by $\mathbf{A}^2$, the resulting vector contains the number of walks of length two from node six to each node; we can reach nodes two, three, four, five, and eight in two moves, and we can return to the original node in three different ways (via nodes five, seven, and eight).

- Node indexing in graphs is arbitrary.
  - ↳ Permuting the node indices permutes the columns of X ; both rows & columns of the adjacency matrix A.
  - ↳ The underlying graph is unchanged.
- Exchanging node indices can be expressed mathematically by a permutation matrix, P.
  - ↳ A matrix where 1 entry in each row & column are 1, the rest are 0.
  - ↳ When position (m, n) of the permutation matrix is set to 1, it indicates node n will become node m after the permutation.
  - ↳ To map from one indexing to another, we use operations:

$$X' = XP$$
$$A' = P^T AP$$

↳ Post multiplying by P permutes the columns.

$A' = P^T A P$ → Post multiplying by P permutes the columns.
Pre multiplying by P^T permutes the rows.

- Any processing applied to the graph should also be indifferent to these permutations.
  - ↳ Otherwise the result will depend on the choice of node indices.

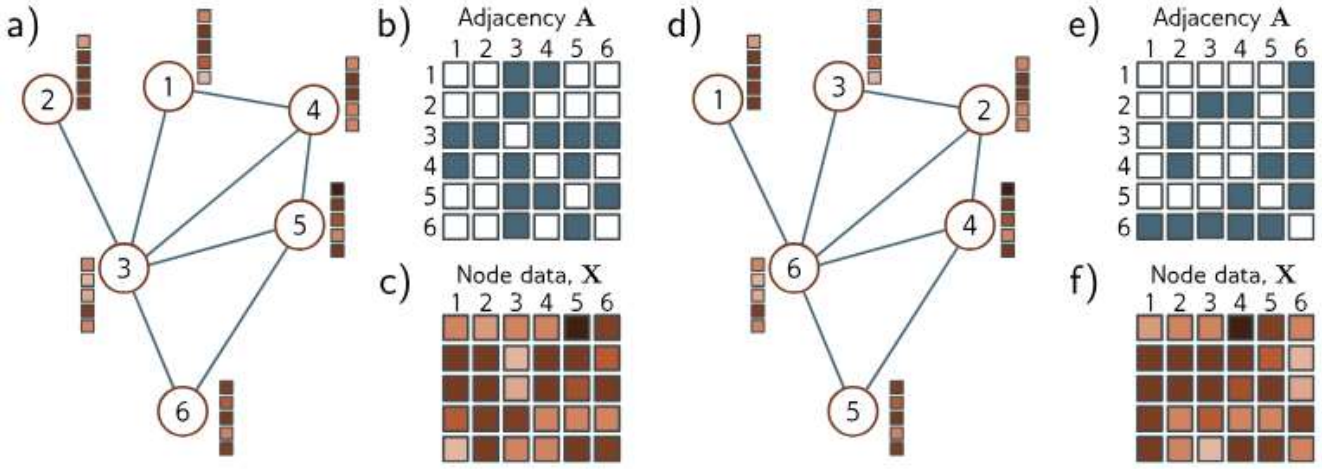


Figure 13.5 Permutation of node indices. a) Example graph, b) associated adjacency matrix and c) node embeddings. d) The same graph where the (arbitrary) order of the indices has been changed. e) The adjacency matrix and f) node matrix are now different. Consequently, any network layer that operates on the graph should be indifferent to the ordering of the nodes.

13.3: Graph Neural Networks, Tasks, & Loss Functions:

- A graph neural network is a model that takes the node embeddings X & the adjacency matrix A as inputs & passes them through K layers.
- The node embeddings are updated at each layer to create intermediate "hidden" representations H_k before finally computing output embeddings H_K.
- At the start of the network, each column of the input node embeddings X just contains information about the node itself.
- At the end, each column of the model output H_K includes information about the node & its context within the graph.

- At the end, each column of the model output H_K includes information about the node & its context within the graph.
  - ↳ Similar to word embeddings passing through a transformer network.
    - ↳ Represent words at the start, but represent word meanings in the context of the sentence at the end.

- Supervised graph problems usually fall into 1 of 3 categories:

1) Graph Level Tasks:

The network assigns a label or estimates one or more values from the entire graph, exploiting both the structure & the node embeddings.

The output node embeddings are combined [e.g by averaging] & the resulting vector is mapped via a linear transformation of neural network to a fixed size vector.

For regression, the mismatch between the result & the ground truth values is computed using least squares loss.

For binary classification, the output is passed through a sigmoid function, & the mismatch is calculated using Binary Cross entropy loss. The probability that the graph belongs to class one might be given by:

$$Pr(y=1|X,A) = sig[\beta_K + \omega_K H_K \bar{1}/N]$$

β_K and ω_K: Learned Parameters

$\bar{1}$: size 1×D

* Post multiplying the output embedding matrix H_K by the column vector 1 has the effect of summing together all the embeddings & subsequently dividing by the number of nodes N computes the average. Known as mean Pooling.

2) Node Level Tasks:

The network assigns a label [classification] or one or more values [regression]

The network assigns a label [classification] or one or more values [regression] to each node of the graph, using both graph structure & node embeddings.

Loss functions are defined in the same way as for graph level tasks, except its now done independently at each node n:

$$Pr(y^{(n)} = 1 | X, A) = sig[\beta_k + w_k h_k^{(n)}]$$

3) Edge Prediction Tasks:

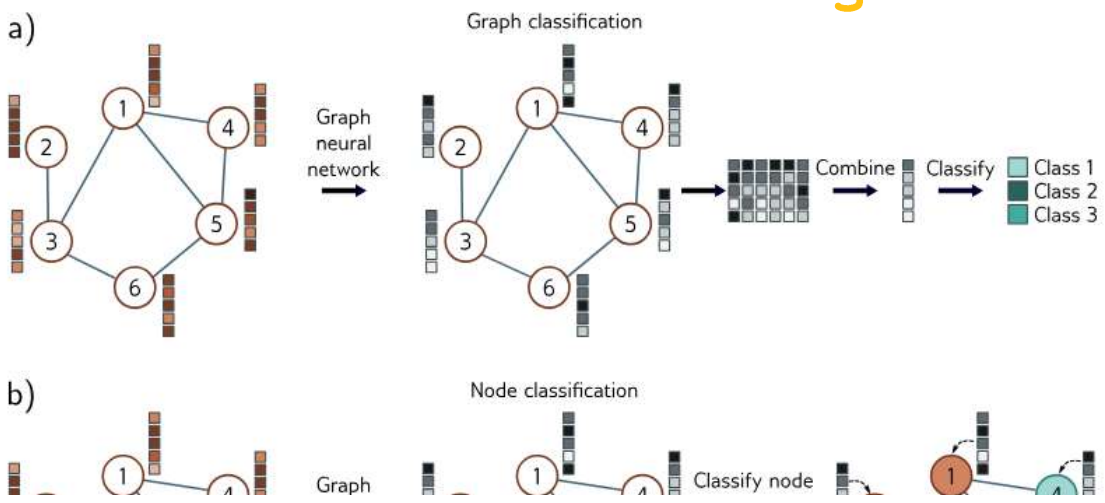
The network predicts whether or not there should be an edge between nodes n & m.

E.g Social networks predict whether 2 users know & like each other & whether they connect.

↳ Binary classification task where the two node embeddings must be mapped to a single number representing the probability that the edge is present.

One possibility is to take the dot product of the node embeddings & pass the result through a sigmoid function to create the probability:

$$Pr(y^{(mn)} = 1 | X, A) = sig[h^{(m)T} h^{(n)}]$$



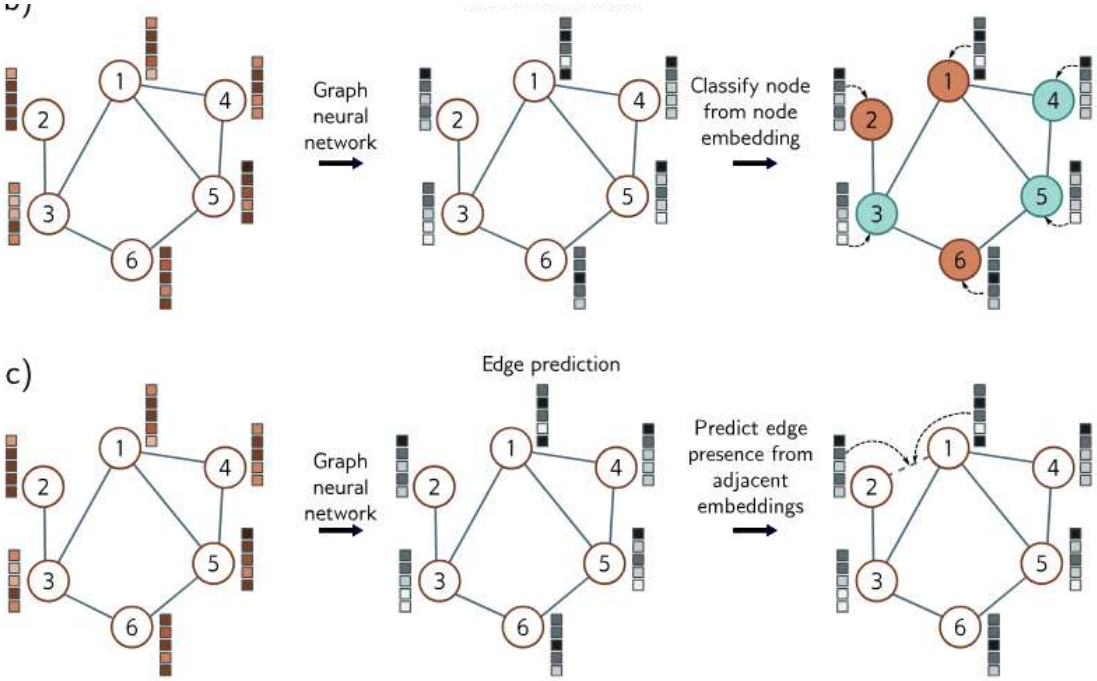


Figure 13.6 Common tasks for graphs. In each case, the input is a graph represented by its adjacency matrix and node embeddings. The graph neural network processes the node embeddings by passing them through a series of layers. The node embeddings at the last layer contain information about both the node and its context in the graph. a) Graph classification. The node embeddings are combined (e.g., by averaging) and then mapped to a fixed-size vector that is passed through a softmax function to produce class probabilities. b) Node classification. Each node embedding is used individually as the basis for classification (cyan and orange colors represent assigned node classes). c) Edge prediction. Node embeddings adjacent to the edge are combined (e.g., by taking the dot product) to compute a single number that is mapped via a sigmoid function to produce a probability that a missing edge should be present.

13.4: Graph Convolutional Networks:

- Spatial based convolutional graph neural networks / GCN.
- Convolutional in that they update each node by aggregating information from nearby nodes.
- They induce a relational inductive bias [bias towards prioritizing information from neighbors].
- They are spatial based because they use the original graph structure.
  - ↳ In contrast to spectral based methods which apply convolutions in

↳ In contrast to spectral based methods which apply convolutions in the Fourier domain.

- Each layer of the GCN is a function $F[\cdot]$ with parameters Φ that take the node embeddings & adjacency matrix & output new node embeddings.

- The network can be written as:

$$H_1 = F[X, A, \Phi_0]$$
$$H_2 = F[H_1, A, \Phi_1]$$
$$H_3 = F[H_2, A, \Phi_2]$$
$$\vdots$$
$$H_K = F[H_{K-1}, A, \Phi_{K-1}]$$

* X is the input
* A is the adjacency matrix
* H_K has the modified node embeddings at the Kth layer
* Φ_K denotes the parameters that map from layer K to layer $K+1$

- Each layer must be equivariant with respect to permutations of the node indices, since node indexing / permutations are arbitrary.

↳ So if we permute the node indices, the node embeddings at each stage will be permuted in the same way.

↳ If P is a permutation matrix, then we must have:

$$P F[X, A] = F[PX, PAP^T]$$

$$H_{K+1}P = F[H_K P, P^T A P, \phi_K]$$

- For node classification & edge prediction tasks, the output should also be equivariant w/ respect to permutations of the node indices.

- For graph level tasks the final layer aggregates information from across the graph so the output is invariant to the node order.

  ↳ The output layer achieves this, since:

  $$y = Sig[B_K + W_K H_K \bar{1}/N] = Sig[B_K + W_K H_K P\bar{1}/N]$$

  for any permutation matrix P.

- Mirrors the case for images, where segmentation should be equivariant to geometric transformations, & image classification should be invariant.

- Here, convolutional & pooling layers partially achieve this w/ respect to translations, but there's no known way to guarantee these properties exactly for more general transformations.

  ↳ But for graphs its possible to define networks that are equivariant or invariant to permutations.

- Like CNNs, we want GCNs to share parameters. Same reasoning applies. The network must independently learn the meanings of the connections in the graph at each node.

  ↳ Training would require many graphs w/ the same topology.

  ↳ Instead, we build a model that uses the same parameters at each node, sharing what the model learns at each node across the entire graph.

- In a CNN & convolution updates a variable by taking a weighted sum of information from its neighbors. In images the neighbors were pixels from a fixed sized region from the

In a CNN, convolutional layers do a similar thing by taking a weighted sum of information from its neighbors. In images the neighbors were pixels from a fixed sized region from the current position, so spatial relationships at each position are the same.

↳ In a graph, each node may have a different # of neighbors

↳ No consistent relationships

↳ So we need to weigh information from nodes differently.

- This leads us to a simple GCN layer.

↳ At each node n in layer K, we aggregate information from neighboring nodes by summing their node embeddings $h_\cdot$:

$$agg[n,K] = \sum_{m \in ne[n]} h_K^{(m)}$$

$ne[n]$ ↳ Returns the set of indices of the neighbors of node n.

↳ We then apply a linear transformation Ω_K to the embedding $h_K^{(n)}$ at the current node, & to the aggregate value. Add a bias term β_K & pass the result through a non linear activation function $a[\cdot]$, applied independently to every member of its vector argument:

$$h_{K+1}^{(n)} = a[\beta_K + \Omega_K h_K^{(n)} + \Omega_K \cdot agg[n,K]]$$

- Can be written more compactly since post multiplication of a matrix by a vector returns the weighted sum of its columns.

↳ The nth column of the adjacency matrix A has 1s at positions of the neighbors.

↳ So if we store the node embeddings into the $D \times N$ matrix H_K & post multiply by the adjacency matrix A, the nth column of the result is $agg[n,K]$:

$$H_{K+1} = a[\beta_K \bar{1}^T + \Omega_K H_K + \Omega_K H_K A]$$
$$= a[\beta_K \bar{1}^T + \Omega_K H_K (A + \bar{I})]$$

↳ $N \times 1$ vector containing 1s.

* The nonlinear activation is applied independently to every member of its matrix argument.

- This layer is equivariant to permutations of the node indices, can cope with any number of layers, exploits the graph structure to provide a relational inductive bias, & shares parameters throughout the graph.

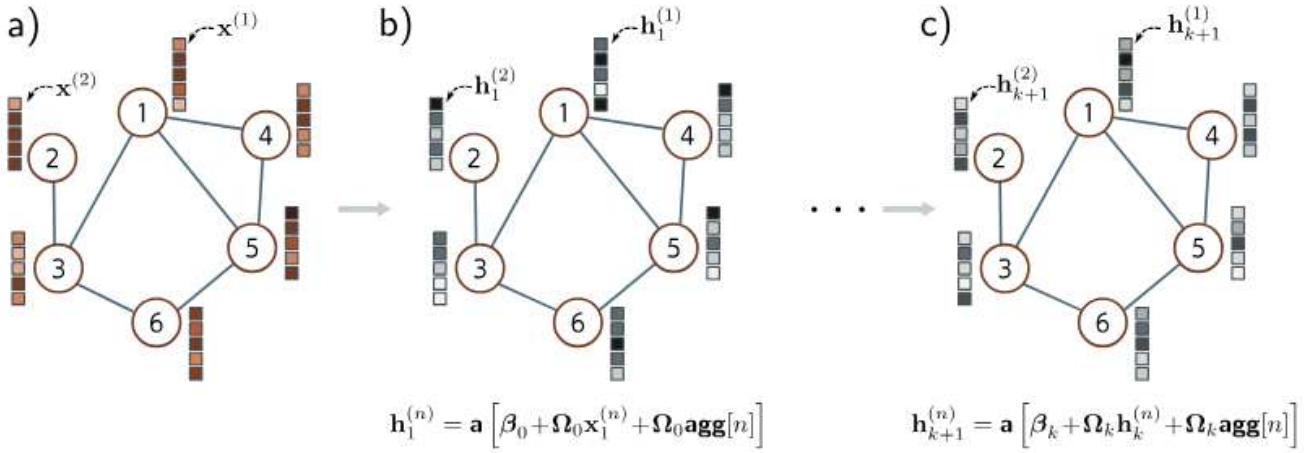


Figure 13.7 Simple Graph CNN layer. a) Input graph consists of structure (embodied in graph adjacency matrix $\mathbf{A}$, not shown) and node embeddings (stored in columns of $\mathbf{X}$). b) Each node in the first hidden layer is updated by (i) aggregating the neighboring nodes to form a single vector, (ii) applying a linear transformation $\boldsymbol{\Omega}_0$ to the aggregated vector, (iii) applying the same linear transformation $\boldsymbol{\Omega}_0$ to the original node, (iv) adding these together with a bias $\boldsymbol{\beta}_0$, and finally (v) applying a nonlinear activation function $\mathbf{a}[\bullet]$ like a ReLU. c) This process is repeated at subsequent layers (but with different parameters for each layer) until we produce the final embeddings at the end of the network.

13.5: Example, Graph Classification:

- We describe a network that classifies molecules as toxic or harmless.

- Network inputs are the adjacency matrix A & node embeddings X.
  - $A \in \mathbb{R}^{N \times N}$ derives from the molecular structure.
  - The columns of the node embedding matrix $X \in \mathbb{R}^{118 \times N}$ are 1-hot vectors indicating which of the 118 elements of the periodic table are present.
  - Node embeddings can be transformed to an arbitrary size D by the first weight matrix $\Omega_0 \in \mathbb{R}^{D \times 118}$.

- Network equations are:

$$H_1 = a[\beta_0 \bar{1}^T + \Omega_0 X(A + \bar{I})]$$

$$H_2 = a[\beta_1 \bar{1}^T + \Omega_1 H_1(A + \bar{I})]$$

$$\vdots$$

$$H_K = a[\beta_{K-1} \bar{1}^T + \Omega_{K-1} H_{K-1}(A + \bar{I})]$$

$$f[X, A, \Phi] = \text{Sig}[\beta_K + \omega_K H_K \bar{1}/N]$$

  - $f[X, A, \Phi]$: A scalar that determines the probability that the molecule is toxic.

- Given I training graphs $\{X_i, A_i\}$ & their labels y_i, the parameters $\Phi = \{\beta_k, \Omega_k\}_{k=0}^{K}$ using SGD & the binary cross entropy loss.

... their labels y_i, the parameters $\Theta = \{\beta_k, \Omega_k\}_{k=0}$ using SGD; the binary cross entropy loss.

- Traditional batching is difficult here, since each graph may have a different # of nodes.
  - ↳ Hence the matrices X_i & A_i have different sizes; there's no way to concatenate them to 3D tensors.
  - ↳ But we can still process the batch in parallel:
    - ↳ The graphs in the batch are treated as disjoint components of a single big graph.
    - ↳ The network can then be run as a single instance of the network equations.
    - ↳ The mean pooling is carried out only over individual graphs to make a single representation per graph that can be fed into the loss function.

13.6: Inductive vs. Transductive Models:

- So far, all the models in the book have been inductive.
  - ↳ Exploit a training set of labeled data to learn the relation between inputs & outputs.
  - ↳ Then applied to the test data.
- A transductive model considers both the labeled & the unlabeled data at the same time.
  - ↳ Doesn't produce a rule, but just a labeling for the unknown outputs.
  - ↳ Also called semi-supervised learning.
- Has the advantage that it can use patterns in the unlabeled data to help make its decisions.
  - ↳ Disadvantage: The model needs to be retrained when extra unlabeled data are added.
- Both problem types are commonly encountered for graphs.
- Ex: A large graph of scientific papers, where labels indicate the academic field. They exist for some nodes, but not all nodes.
  - ↳ Here, the training & testing data are irrevocably connected.
- Graph level tasks only occur in the inductive setting where there are training &

- Graph level tasks only occur in the inductive setting where there are training & test graphs.
  - ↳ But node level tasks & edge prediction tasks can occur in either setting!
- In the transductive case, the loss function minimizes the mismatch between the model output & the ground truth where this is known.
  - ↳ New predictions are computed by running the forward pass & retrieving the results where the ground truth is unknown.

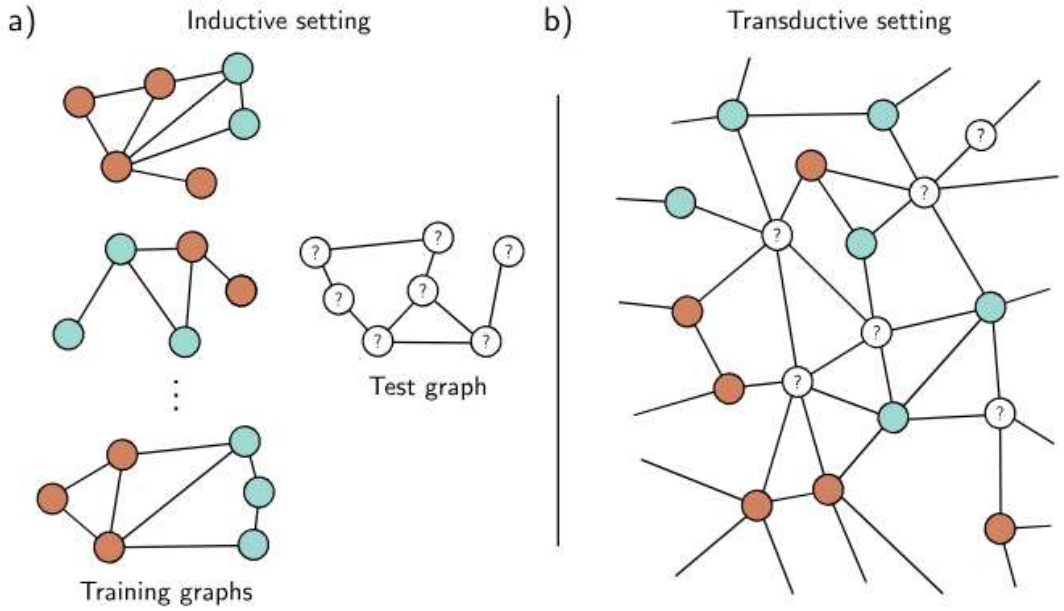


Figure 13.8 Inductive vs. transductive problems. a) Node classification task in the inductive setting. We are given a set of I training graphs, where the node labels (orange and cyan colors) are known. After training, we are given a test graph and must assign labels to each node. b) Node classification in the transductive setting. There is one large graph in which some nodes have labels (orange and cyan colors), and others are unknown. We train the model to predict the known labels correctly and then examine the predictions at the unknown nodes.

13.7: Node Classification:

- Consider binary node classification in the transductive setting.
- Start w/ a graph w/ millions of nodes.
  - ↳ Some nodes have ground truth binary labels.
  - ↳ Goal is to label the unlabeled nodes.

- The body of the network will be the same as the previous example, but w/ a different final layer that produces an output vector of size $1 \times N$:

$$f[X, A, \Phi] = \text{Sig}[\beta_k \vec{1}^T + W_k H_k]$$

↳ Applies the sigmoid independently to every element of the row vector input.

- We use binary cross entropy loss, but only at nodes where we know the ground truth label y.

- There are 2 problems w/ training this network.

1) Difficult to train a graph neural network of this size.
   ↳ we have to store the node embeddings at every layer in the forward pass.

2) We only have 1 graph, so how do we do SGD?
   ↳ How can we form a batch if there's only 1 object?

- One way to form a batch is to choose a random subset of labeled nodes nodes at each training step.
  ↳ Each node depends on its neighbors in the previous layer.
  ↳ These in turn depend on its neighbors in the layer before.
  ↳ So each node has a receptive field [just like CNNs]
  ↳ The receptive field region is called the K-hop neighborhood.

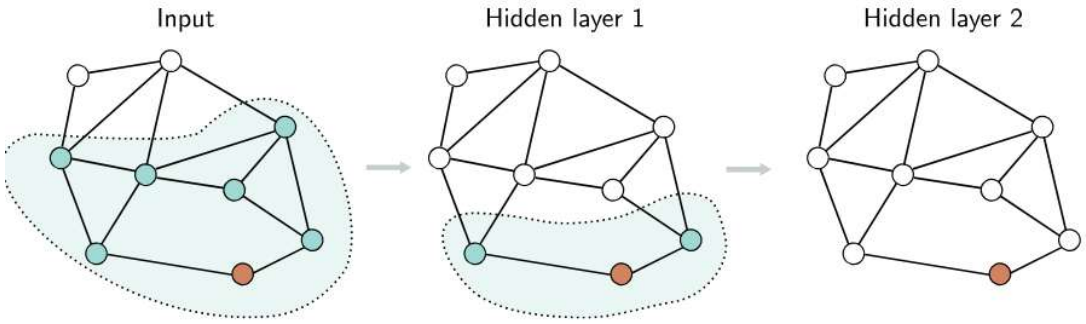


Figure 13.9 Receptive fields in graph neural networks. Consider the orange node in hidden layer two (right). This receives input from the nodes in the 1-hop neighborhood in hidden layer one (shaded region in center). These nodes in hidden layer one receive inputs from their neighbors in turn, and the orange node in layer two receives inputs from all the input nodes in the 2-hop neighborhood (shaded area on left). The region of the graph that contributes to a given node is equivalent to the notion of a receptive field in convolutional neural networks.

- We can perform a gradient descent step using the graph that forms the union of the K-hop neighborhoods of the batch nodes.
  - ↳ The remaining nodes don't contribute.
- If there are too many layers ! the graph is densely connected, every input node may be in the receptive field of every output, ; this may not reduce the graph size at all.
  - ↳ known as the graph expansion problem.
  - ↳ 2 approaches to tackle the problem are neighborhood sampling ; graph Partitioning.
- Neighborhood Sampling:

The full graph that feeds into the batch of nodes is sampled, reducing the connections at each network layer.

Ex:

Start w/ the batch nodes ; randomly sample a fixed number of their neighbors in the previous layer.

Then we randomly sample a fixed number of their neighbors in the layer before, ; so on.

The graph still increases in size w/ each layer but in a more controlled way.

This is done again for each batch, so the contributing neighbors differ even if the same batch is drawn twice.

Similar to dropout, ∴ adds regularization.

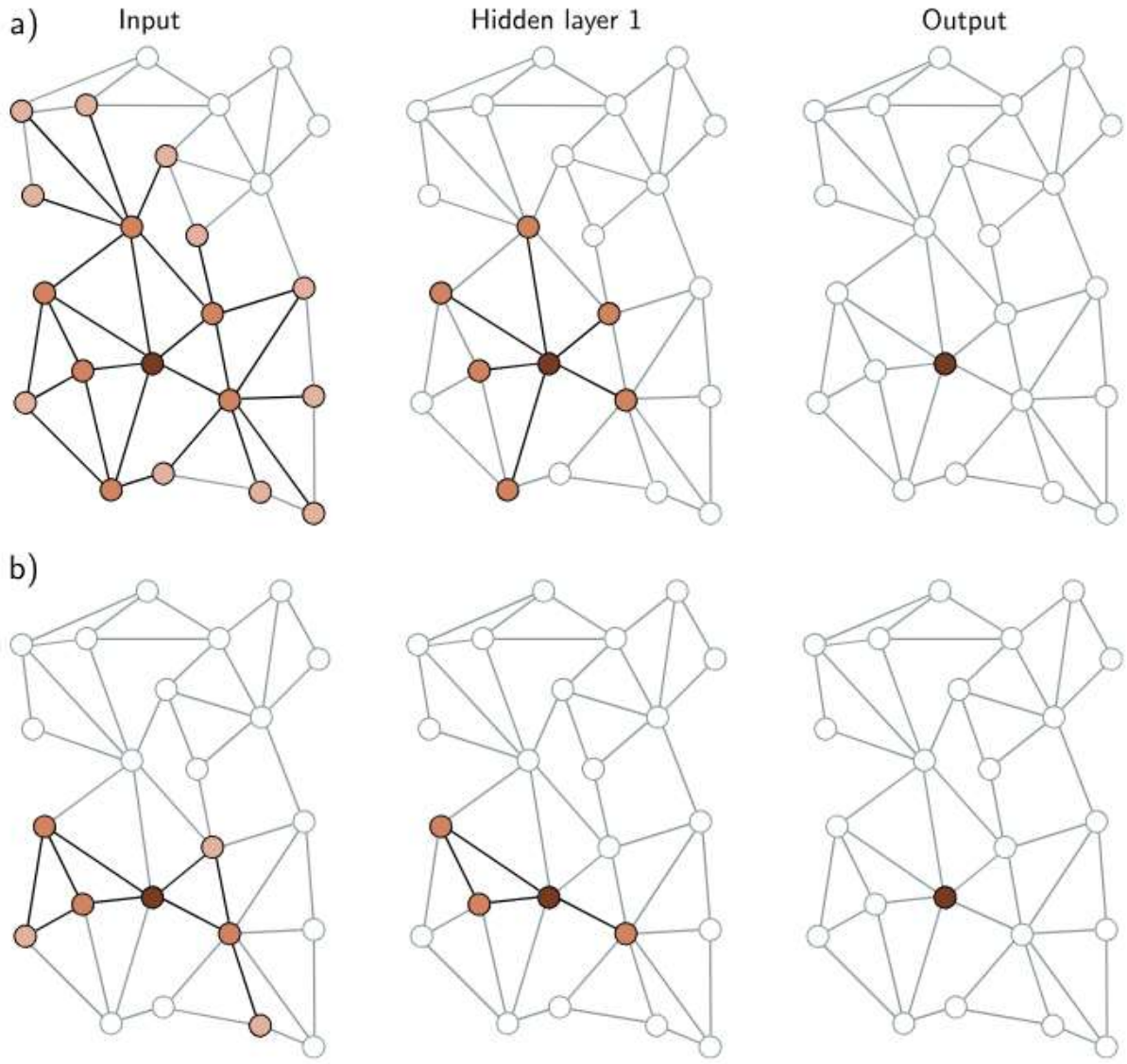


Figure 13.10 Neighborhood sampling. a) One way of forming batches on large graphs is to choose a subset of labeled nodes in the output layer (here, just one node in layer two, right) and then working back to find all of the nodes in the K-hop neighborhood (receptive field). Only this sub-graph is needed to train this batch. Unfortunately, if the graph is densely connected, this may retain a large proportion of the graph. b) One solution is neighborhood sampling. As we work back from the final layer, we select a subset of neighbors (here, three) in the layer before and a subset of the neighbors of these in the layer before that. This restricts the size of the graph for training the batch. In all panels, the brightness represents the distance from the original node.

- Graph Partitioning:

- Graph Partitioning:

Cluster the original graph into disjoint subsets of nodes before processing.

Algorithms exist to choose the subsets to maximize the # of internal links.

The smaller graphs can each be treated as batches, or a random subset of them can be combined to form a batch [by reinstating any edges between them from the original graph].

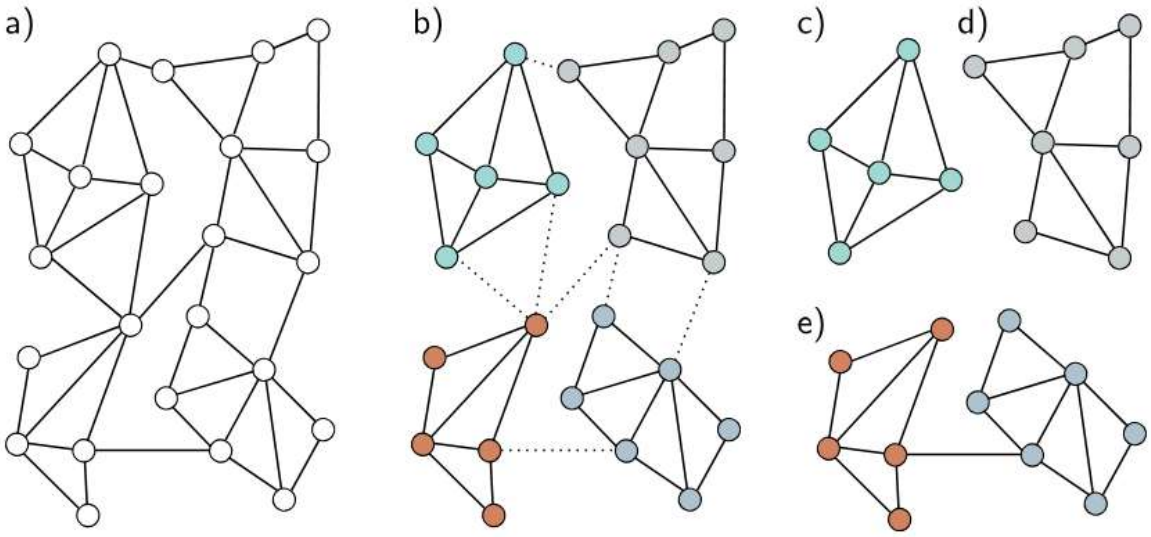


Figure 13.11 Graph partitioning. a) Input graph. b) The input graph is partitioned into smaller subgraphs using a principled method that removes the fewest edges. c-d) We can now use these subgraphs as batches to train in a transductive setting, so here, there are four possible batches. e) Alternatively, we can use combinations of the subgraphs as batches, reinstating the edges between them. If we use pairs of subgraphs, there would be six possible batches here.

- Using one of the 2 methods above, we can train a network in the same way as the inductive setting, dividing the labeled nodes into train, test, & validation sets.

- For inference, we compute predictions for the unknown nodes based on their K-hop neighborhood.
  - ↳ Doesn't require storing the intermediate representations.
  - ↳ Much more efficient than training.

↳ Much more efficient than training.

13.8: Layers for graph convolutional networks:

- We can consider different approaches to both:
  - ↳ The combination of the current embedding w/ the aggregated neighbors.
  - ↳ The aggregation process itself
- In the standard GCN layer, we combined aggregated neighbors HA w/ the current nodes H by summing them:

$$H_{k+1} = a\left[\beta_k \vec{1}^T + \Omega_k H_k (A + I)\right]$$

- In another variation the current node is multiplied by a factor of $(1+\varepsilon_k)$ before contributing to the sum. ε_k is a learned scalar that's different for each layer:

$$H_{k+1} = a\left[\beta_k \vec{1}^T + \Omega_k H_k (A + (1+\varepsilon_k) I)\right]$$

↳ Known as diagonal enhancement.

- A related variation applies a different linear transform Ψ_k to the current node:

$$\begin{aligned} H_{k+1} &= a\left[\beta_k \vec{1}^T + \Omega_k H_k A + \Psi_k H_k\right] \\ &= a\left[\beta_k \vec{1}^T + \begin{bmatrix}\Omega_k & \Psi_k\end{bmatrix}\begin{bmatrix}H_k A \\ H_k\end{bmatrix}\right] \\ &= a\left[\beta_k \vec{1}^T + \Omega'_k \begin{bmatrix}H_k A \\ H_k\end{bmatrix}\right] \end{aligned}$$

$$\Omega'_k = \begin{bmatrix}\Omega_k & \Psi_k\end{bmatrix}$$

- We can also incorporate residual connections. Here, the aggregated representation from the

Neighbors is transformed & passed through the activation function before summation / concatenation w/ the current node.

↳ For the latter case:

$$H_{k+1} = \begin{bmatrix} a[\beta_k 1^T + \Omega_k H_k A] \\ H_k \end{bmatrix}$$

- So far we've aggregated the neighbors by summing the node embeddings.
  - ↳ we can aggregate in different ways.
  - ↳ Sometimes it's better to take the average than the sum.
    - Could be better if the embedding information is more important than the structural information
      - ↳ since the magnitude of neighborhood contributions doesn't depend on the # of neighbors.

$$agg[n] = \frac{1}{|ne[n]|} \sum_{m \in ne[n]} h_m$$

↳ A set containing the indices of the neighbors of the nth node.

Can be computed neatly in matrix form by introducing the diagonal NxN degree matrix, D. Each nonzero element of this matrix contains the number of neighbors for the associated node.

Each diagonal element in the inverse matrix D^{-1} contains the denominator needed to compute the average. The new GCN layer can be written as:

$$H_{k+1} = a[\beta_k 1^T + \Omega_k H_k (AD^{-1} + I)]$$

- There are many variations based on mean aggregation.
  - ↳ Sometimes the current node is included in the aggregation process.
  - ↳ Kipf normalization:

$$agg[n] = \sum_{m \in ne[n]} \frac{h_m}{\sqrt{|ne[n]||ne[m]|}}$$

Logic: Information coming from nodes w/ a large # of neighbors should be down weighted. Since there's many connections & provide less unique information.

↳ can be expressed with the degree matrix:

$$H_{k+1} = a[B_k 1^T + \Omega_k H_k (D^{-1/2} A D^{-1/2} + I)]$$

- An alternative method invariant to permutation is computing the maximum of a set of objects.
  - ↳ Max Pool Aggregation:

$$agg[n] = \max_{m \in ne[n]} [h_m]$$

  - * Max[·] returns the element wise maximum of the vectors h_m that are neighbors to the current node n.

- So for all aggregation methods weigh the contribution of the neighbors equally or in a way that depends on the graph topology.
  - ↳ In graph attention, the weights depend on the data at the nodes.
  - ↳ A linear transformation is applied to the current node embeddings so that:

$$H'_k = B_k 1^T + \Omega_k H_k$$

Then the similarity s_{mn} of each transformed node embedding h'_m to the transformed node embedding h'_n is computed by concatenating the pairs, taking a dot product w/ a column vector ϕ_k of learned parameters, & applying an activation function:

$$s_{mn} = a\left[\phi_k^T \begin{bmatrix} h'_m \\ h'_n \end{bmatrix}\right]$$

  - ↳ These are stored in an $N \times N$ matrix S. Each element represents the similarity of every node to every other.
  - ↳ The attention weights are normalized to be positive & sum to one using Softmax.
  - ↳ Only those values corresponding to the current node & its neighbors should contribute.
  - ↳ The attention weights are applied to the transformed embeddings:

$$H_{n+1} = a[H'_k \cdot \text{Softmax}[S, A+I]]$$

$H_{k+1} = a[H'_k \cdot \text{Softmask}[S, A+I]]$

↳ A 2nd activation function.

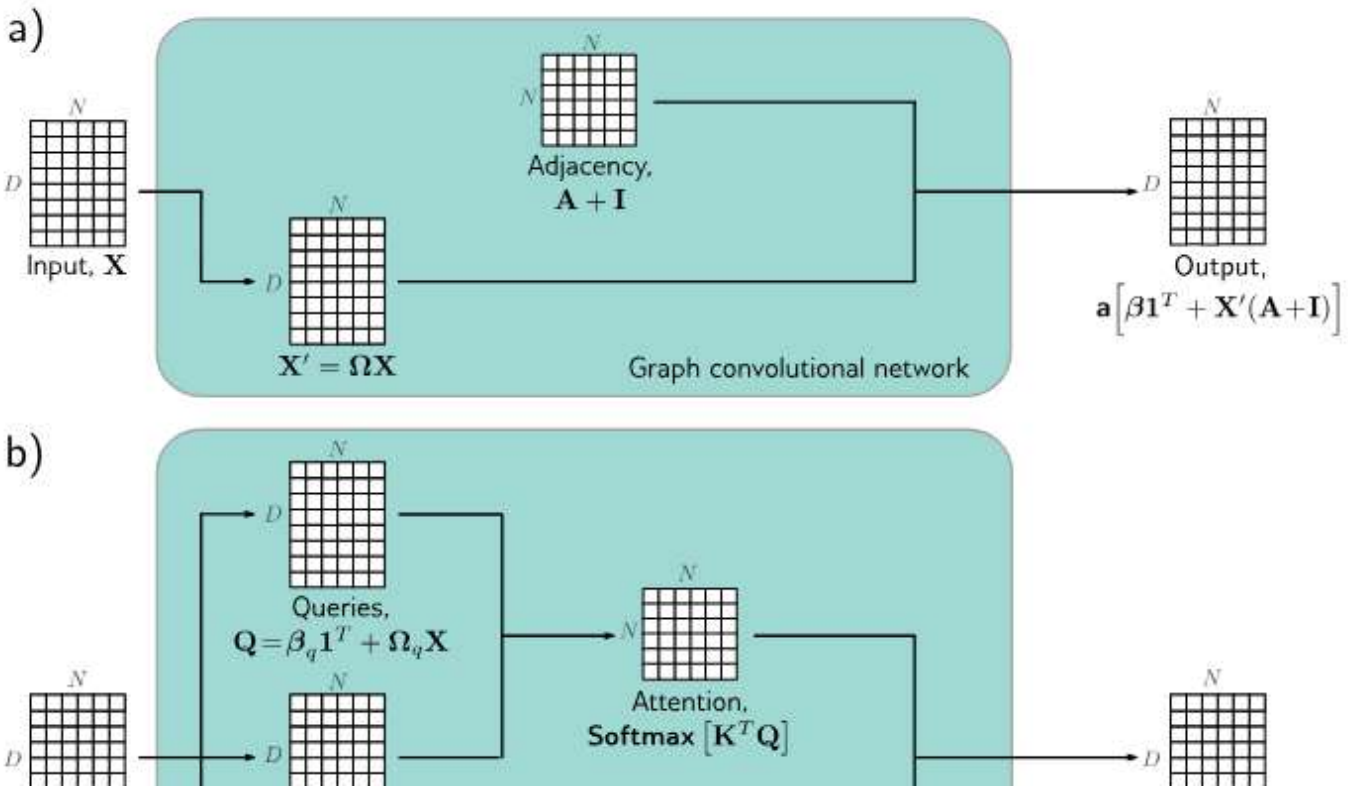
↳ Computes attention values by applying a softmax operation seperately to each column of its first argument, only after setting values where the second argument $A+I$ is 0 to $-\infty$, so they dont contribute.

Ensures the attention to non neighboring nodes is 0.

*Similar to dot product self attention except:

1) Keys, queries, values are all the same
2) The measure of similarity is different
3) The attentions are masked so that each node only attends to itself & its neighbors.

*Like transformers, we can use multiple heads that are run in parallel & recombined.



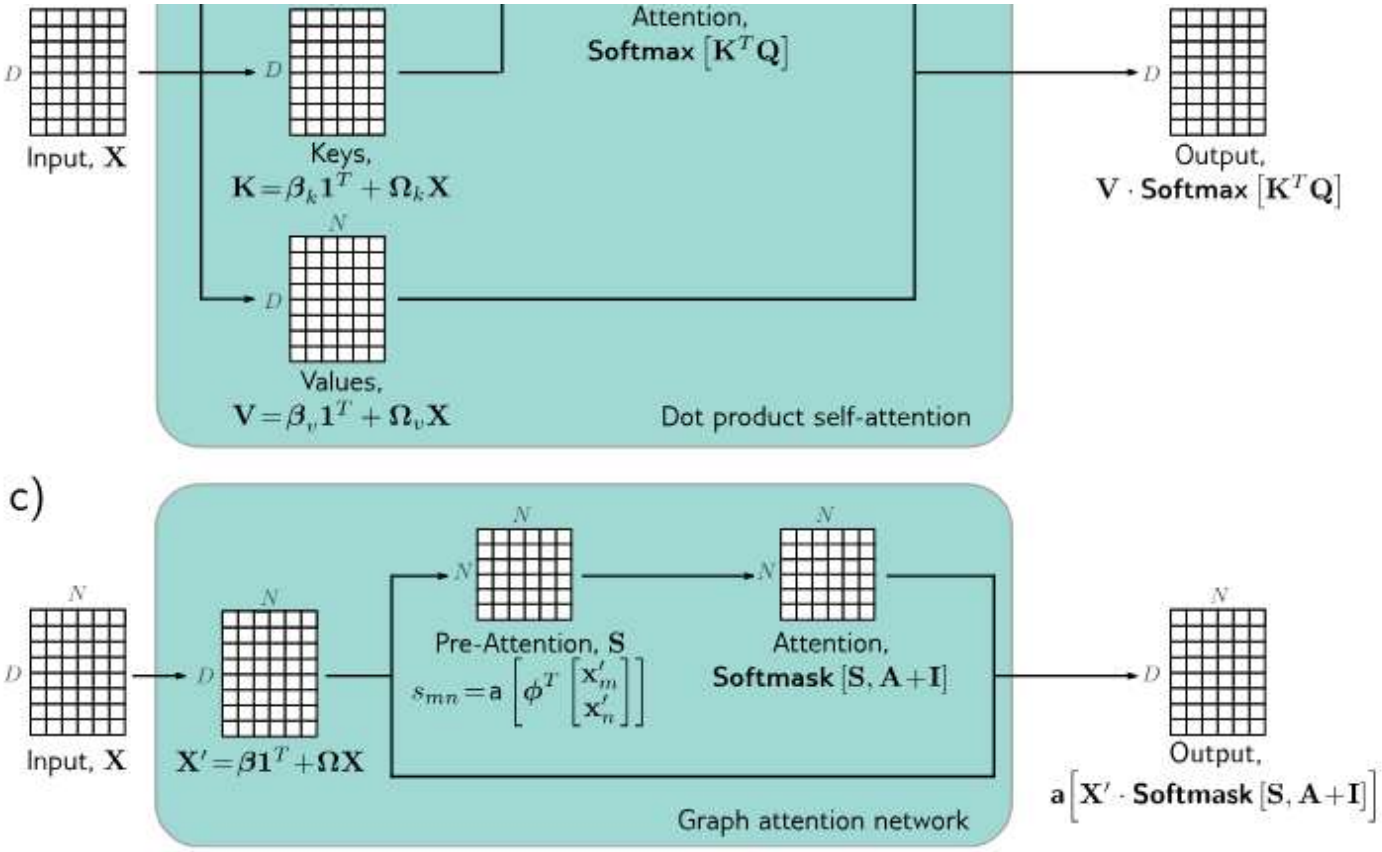


Figure 13.12 Comparison of graph convolutional network, dot product attention, and graph attention network. In each case, the mechanism maps N embeddings of size D stored in a $D \times N$ matrix $\mathbf{X}$ to an output of the same size. a) The graph convolutional network applies a linear transformation $\mathbf{X}' = \mathbf{\Omega X}$ to the data matrix. It then computes a weighted sum of the transformed data, where the weighting is based on the adjacency matrix. A bias β is added, and the result is passed through an activation function. b) The outputs of the dot-product self-attention mechanism in the transformer are also weighted sums of the transformed inputs, but this time the weights depend on the data itself via the attention matrix. c) The graph attention network combines both of these mechanisms; the weights are both computed from the data and based on the adjacency matrix.

13.9: Edge Graphs:

- We now consider the case where the information is associated w/ the edges of the graph.
- We can adapt the model for node embeddings to process edge embeddings using the edge graph.
  - ↳ Each edge of the original graph becomes a node
  - ↳ Every 2 edges w/ a common node in the original graph create an edge in the new graph.
  - ↳ Generally a graph can be recovered from its edge graph, so it's possible to swap between the two.

... edge ... common node ... in the original graph ... create an edge in the new graph.

↳ Generally a graph can be recovered from its edge graph, so it's possible to swap between the two.

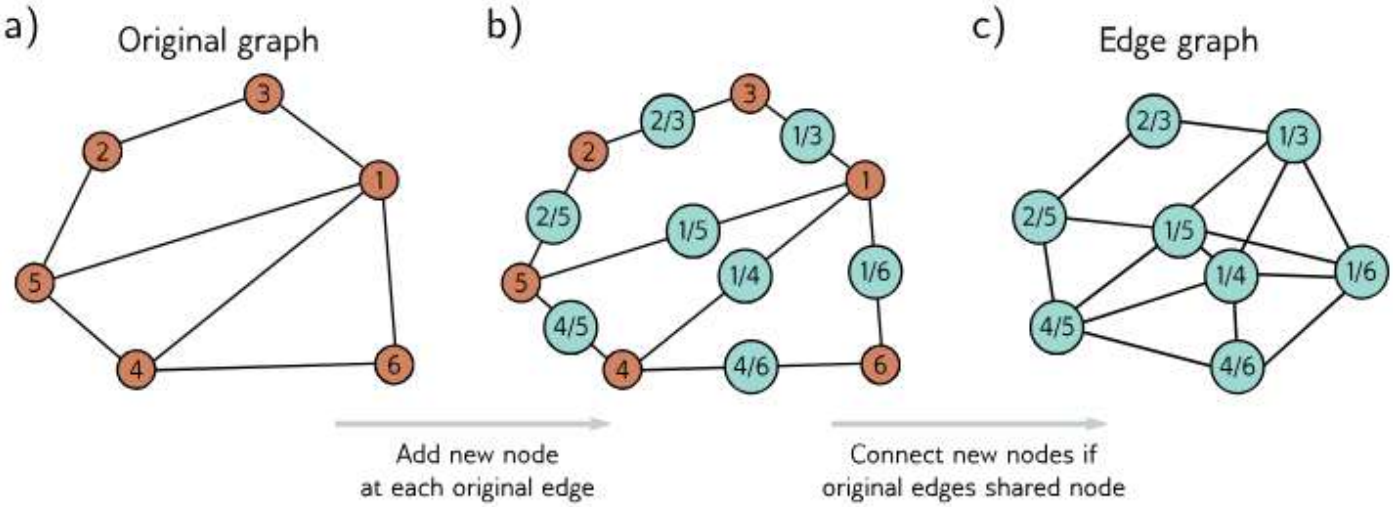


Figure 13.13 Edge graph. a) Graph with six nodes. b) To create the edge graph, we assign one node for each original edge (cyan circles), and c) connect the new nodes if the edges they represent connect to the same node in the original graph.

- To process edge embeddings, we translate the graph to the edge graph.
  ↳ Then we use the exact same techniques as above.

- When both node & edge embeddings are present, we can translate back & forth between representations.
  ↳ So there are 4 possible updates:
  1) Node updates node
  2) Node updates edge
  3) Edge updates node
  4) Edge updates edge

↳ These can be alternated as desired, or [with some modification] nodes can be updated simultaneously from both nodes & edges.

13.10: Summary:

- Graphs consist of a set of nodes, connected by edges.

- Graphs consist of a set of nodes, connected by edges.
- Both nodes & edges can store data, known as an embedding.
- Tasks: Graph-level, node-level, or edge-level!
- GNNs are deep learning models applied to graphs. Since the node order of graphs is arbitrary the layers of a GNN must be equivariant to permutations of the node indices.
- Spatial based convolutional networks aggregate information from neighbors of a node; then use this to update the node embeddings.
- One challenge is that they appear in transductive settings, where there's only one partially labeled graph instead of training / testing sets. This led to sampling; partitioning algorithms.
- To process edge embeddings, we can convert a graph to an edge graph; use GNNs.